מעבדה 3: מחזור חיי התהליך וניהולו ב - Linux

הקדמה ומטרות

מעבדה זו מתמקדת ביצירה וניהול של תהליכים במערכת ההפעלה. נלמד כיצד תהליך אב יוצר תהליך בן, כיצד הם פועלים במקביל, וכיצד האב מסנכרן את סיומם. **מטרות המעבדה:**

- הבנת מנגנון השכפול של תהליכים באמצעות $fork()$.
- שימוש ב $wait()$ & $waitpid()$ לסנכרון ומניעת תהליכי "Zombie".
- החלפת מרחב הכתובות של תהליך באמצעות משפחת פונקציות $exec()$ (דוגמאות משפחה נפצות מופיעות בנספח).
- הבנת ההבדל בין קוד האב לקוד הבן על בסיס ערך החזרה של ה – PID.

דגשים והסברים (למידה עצמאית ודוגמאות)

תהליך אינו רק קוד, הוא יחידה פעילה הכוללת את מצב הרכיבים (Program Counter, רגיסטרים) ומרחב כתובות בזיכרון. המערכת מנהלת כל תהליך באמצעות מבנה נתונים הנקרא PCB (Process Control Block), המכיל את ה PID (מזהה ייחודי), מצב התהליך (Ready, Running, וכו') ומידע על ניהול הזיכרון שלו.

יצירת תהליך עם $fork()$

כאשר נקראת הפונקציה $fork()$ נוצר תהליך בן שהוא עותק של האב. שניהם ממשיכים לבצע את הקוד מהנקודה שמיד לאחר הקריאה ל $fork$

- **בידוד זיכרון**: למרות שהבן הוא עותק, יש לו מרחב כתובות נפרד, שינוי משתנה בבן לא ישפיע על האב.
- **זיהוי בתוך הקוד**: המפתח להפרדת הלוגיקה הוא ערך החזרה של הקריאה ל $fork$
  - $pid\ == 0$ אתה בתוך הבן.
  - $pid\ > 0$ אתה בתוך האב (הערך הוא ה – PID של הבן).
  - $pid\ == -1$ היצירה נכשלה.

```
pid_t pid = fork();
if (pid == 0) {
    printf("I am the child\n");
} else if (pid > 0) {
    printf("I am the parent of %d\n", pid);
}
```

סנכרון וסיום: $wait()$ & $exit()$

הפונקציה – $exit(int\ status)$ סיום התהליך

כאשר תהליך מסיים את תפקידו, הוא קורא ל $exit$ הפונקציה מקבלת ערך שלם (status) המועבר למערכת ההפעלה.

- **שחרור משאבים:** המערכת משחררת את רוב המשאבים של התהליך (זיכרון, קבצים פתוחים), אך שומרת רשומה קטנה בטבלת התהליכים (PCB) הכוללת את ה PID ואת קוד הסיום.
- **מצב Zombie :** בשלב זה, התהליך נמצא במצב "זומבי" – (Terminated) הוא לא רץ, אך הוא עדיין קיים בטבלה עד שאביו יאסוף את הנתונים שלו.

הפונקציה $wait(\&status)$ – תפקיד האב

הפונקציה $wait$ משמשת לשתי מטרות מרכזיות:

- **סנכרון (Blocking)** - היא גורמת לתהליך האב להפסיק את ריצתו ולעבור למצב "ממתין" (BLOCKED) עד שאחד מבניו יסיים את ריצתו. אם בן כבר סיים והוא במצב זומבי, ה $wait$ חוזר מיד.
- **איסוף נתונים וניקוי** - ברגע שהבן מסיים, ה $wait$ מחזיר לאב את ה PID של הבן שסיים ומעדכן את המשתנה $status$ בקוד הסיום של הבן. רק לאחר פעולה זו, המערכת מוחקת לחלוטין את רשומת הבן מה-PCB.

מקרי קצה

- **תהליכים יתומים (Orphan):** אם האב מסתיים לפני הבן, הבן הופך ל"יתום". במקרה כזה, תהליך ה $init$ או $systemd$ מאמץ אותו ומבצע עבורו $wait$ באופן אוטומטי כשזה יסתיים.
- **מניעת זומבים :**אם אב יוצר בנים רבים ולא קורא ל $wait$, טבלת התהליכים תתמלא ב"זומבים", מה שעלול למנוע יצירת תהליכים חדשים במערכת.

```c
#include <stdio.h>
#include <unistd.h>     // for בשביל
#include <sys/wait.h>   // wait-בשביל פונקציית ה
#include <stdlib.h>     // exit בשביל
int main() {
    int status;
    pid_t pid = fork();
    if (pid == 0) {
        // קוד הבן
        printf("Child: Working...\n");
        sleep( seconds: 2);
        exit( status: 5); // 5 הבן מסיים ושולח סטטוס
    } else {
        // קוד האב
        printf("Parent: Waiting for child %d to finish...\n", pid);
        wait(&status); // exit האב עוצר כאן עד שהבן יבצע
        printf("Parent:Child finished with status %d. Now it's cleared from system.\n",
                WEXITSTATUS(status));
    }
    return 0;
}
```

משימות לביצוע

משימה 1 – The Fork Pair

במשימה זו נראה שלמרות שהבן הוא עותק של האב, יש להם מרחב כתובות נפרד.

1. צרו קובץ בשם $task1_fork.c$
2. בתוך פונקציית ה $main$
   - הגדירו משתנה $x = 100$
   - בצעו $fork()$
   - משימת הבן: תגדיל את x ב – 50 ותדפיס את ערכו ואת ה PID של הבן.
     $My\ PID\ is < pid >: x =< value\ of\ x >$
   - משימת האב: תמתין לסיום הבן (מומלץ להשתמש ב $wait(NULL)$ ולאחר מכן תדפיס את ערכו של x ואת ה PID של האב. $My\ PID\ is < pid >: x =< value\ of\ x >$

משימה 2 – Command Runner using Exec

במשימה זו נלמד כיצד להריץ תוכנית חיצונית מתוך הקוד שלנו.

1. צרו קובץ בשם $task2_exec.c$.
2. התוכנית צריכה לקבל ארגומנט משורת הפקודה (למשל $/task_2\ ls$
3. השתמשו ב $fork()$
   - משימת הבן: תחליף את קוד הביצוע של בפקודה שהתקבלה ב arg [1] באמצעות $execlp()$

- משימת האב: תדפיס Parent: waiting for command to finish, תחכה למשימת הבן לסיום ריצתה ואז תדפיס $Parent: execution\ cpmplete.$

4. נסו להריץ פקודה שלא קיימת ובדקו אם הבן מדפיס שגיאה.

משימה 3 – Process Chain – Vertical Hierarchy

משימה זו דורשת הבנה של לולאות וסנכרון היררכי.

1. צרו קובץ בשם $task3_chain.c$.
2. התוכנית תקבל מספר שלם N ותיצור שרשרת של N תהליכים (אב -> בן -> נכד -> ...)
3. **הלוגיקה:**
   - בכל איטרציה של לולאה, התהליך הנוכחי יוצר בן.
   - האב חייב להמתין לבנו ($wait$) ורק אז לסיים.
   - כל תהליך ידפיס. $Process\ [PID], my\ parent\ is\ [PPID]$

- **דגש חשוב:** עליכם לוודא שהתהליכים מסתיימים בסדר הפוך ליצירתם (האחרון בשרשרת מסיים ראשון, ואז זה שיצר אותו, עד לאב המקורי).

משימה 4 – The Multi-Process Calculator

במשימה זו ניצור תהליך אב שמפצל עבודה בין שני בנים שונים במקביל.

1. צרו קובץ בשם $task4_calc.c$
2. התוכנית תקבל שני מספרים מהמשתמש
3. התוכנת תבצע $fork()$ פעמיים כדי ליצור שני בנים:
   - בן א': יחשב את הסכום של שני המספרים ידפיס את התוצאה $child\ 1\ (sum): < num1 > +< num2 > = < sum >$ ויסיים עם קוד יציאה של התוצאה.
   - בן ב': יחשב את המכפלה של המספרים, ידפיס את התוצאה $child\ 1\ (mul): < num1 > +< num2 > = < mul >$ ויסיים עם קוד היציאה של התוצאה.
4. משימת האב: תשתמש ב $waitpid()$ כדי להמתין לכל בן ספציפית לפי ה – PID שלו. לאחר שכל אחד מסיים, האב ידפיס $All\ computations\ done$

- השתמשו במאקרו $WEXITSTATUS(status)$ כדי שגם האב ידפיס בעצמו את התוצאות שחזרו מהבנים. $Parent: child < child\ number > result\ was < result >$

פלט לדוגמא עבור שני המספרים 5, 7

```
Child 1 (Sum): 5 + 7 = 12
Parent: Child 1 result was 12
Child 2 (Mul): 5 * 7 = 35
Parent: Child 2 result was 35
All computations done
```

משימה 5 – Process Race & Zombie Protection

משימה זו בוחנת את השפעת המתזמן על סדר הריצה ואת הטיפול בזומבים.

1. צרו קובץ $task5_race.c$
2. התוכנית תקבל מספר שלם K
3. משימת האב תיצור K בנים בלולאה.

4. **כל בן :**יבצע $sleep(rand() \% 5)$ (זמן שינה אקראי) כדי לדמות עבודה, ואז ידפיס את ה PID שלו וייצא.
   - $child < PID > exiting \ldots$
5. **האב :**מיד לאחר יצירת כל הבנים (בלי לחכות באמצע), יבצע פקודת מערכת $system("ps\ -l")$
   - האב ידפיס $Parent: Reviewing\ process\ state\ (lookk\ for < defunc >)$
6. עליכם לזהות בפלט של ps את התהליכים שנמצאים במצב **Zombie** (מסומנים ב Z או <defunct>).
7. לאחר מכן, האב יבצע לולאת $wait$ כדי לנקות את כל הבנים ויוודא שהם נעלמו מהמערכת ואז ידפיס:
   - $Parent: all\ children\ reaped$

- **מטרה :**להבין שתהליך שסיים ולא נאסף על ידי האב נשאר "תקוע" במשאבי המערכת.

מה להגיש ובאיזה פורמט

יש להגיש קובץ ZIP בשם $Lab3_ID.zip$ (החליפו את ID בתעודת הזהות שלכם) על הקובץ להכיל את הקבצים:

1. $task_fork.c$
2. $task2_excec.c$
3. $task3_chain.c$
4. $task4_calc.c$
5. $task5_race.c$
6. $Makefile$

<u>דגשים להגשה:</u>

- יש לוודא שלכל הקבצים יש הרשאות ריצה.(chmod +x)
- הבודק האוטומטי משווה פלטים בצורה מדויקת .(Case Sensitive) אין להדפיס הודעות נוספות מעבר לנדרש.
- וודאו שכל סקריפט מתמודד נכון עם ארגומנטים חסרים.

אי הקפדה על הנחיות, כולל פורמט הגשה הדיגיטלי, תגרור הורדה בציון התרגיל.

נספח: משפחת פונקציות ה $excec$

כל הפונקציות במשפחה זו מבצעות את אותה פעולה בסיסית :החלפת תהליך קיים בתוכנית חדשה .ההבדלים ביניהן מסומנים על ידי האותיות המתווספות לשם הפונקציה:

- l (List) הארגומנטים מועברים כרשימה (מופרדים בפסיקים).
- v (Vector) הארגומנטים מועברים כמערך של מחרוזות$(char * argv[])$
- p (Path) המערכת תחפש את הקובץ בנתיבי ה $PATH$
- e (Environment) ניתן להעביר משתני סביבה מותאמים אישית.

טבלת השוואה - פונקציות $exec$ נפוצות

מתי נשתמש?	חתימת הפונקציה (פשטנית)	חיפוש ב-PATH?	סוג העברת ארגומנטים	שם הפונקציה
כשיודעים את כמות הארגומנטים מראש ואת הנתיב המדויק.	$execl(path, arg0, \ldots, NULL)$	לא	רשימה $(arg0, arg1\ldots)$	$execl$
הכי נוחה להרצת פקודות shell רגילות כמו ls או date	$execlp(file, arg0, \ldots, NULL)$	כן	רשימה $(arg0, arg1\ldots)$	$execlp$
כשיש לנו מערך מוכן (למשל זה שקיבלנו מה main	$execv(path, argv)$	לא	מערך $(char * argv[])$	$execv$
נפוצה מאוד במימוש של Shell כשהפקודה והארגומנטים דינמיים.	$execvp(file, argv)$	כן	מערך $(char * argv[])$	$execvp$
רמת השליטה הגבוהה ביותר (כולל משתני סביבה). זו קריאת המערכת ה"אמיתית."	$execve(path, argv, envp)$	לא	מערך $(char * argv[])$	$execve$